

Distributed Cache System Inspired by Redis

1. Abstract

Modern large-scale applications require fast access to frequently used data such as user sessions, API responses, and analytics counters. Accessing this data directly from disk-based databases can introduce latency due to disk I/O and complex queries. To address this issue, systems often use in-memory caching solutions like Redis that provide sub-millisecond access times.

This project aims to design and implement a simplified distributed caching system inspired by Redis to demonstrate core distributed systems principles. The system will implement a distributed in-memory key-value store supporting data partitioning (sharding), replication, cache eviction policies, and client APIs. Data will be distributed across multiple nodes using consistent hashing, while replica nodes will ensure high availability in case of node failures.

2. Objectives

The main objective of this project is to design and implement a distributed caching system that demonstrates the core principles used in modern in-memory data stores.

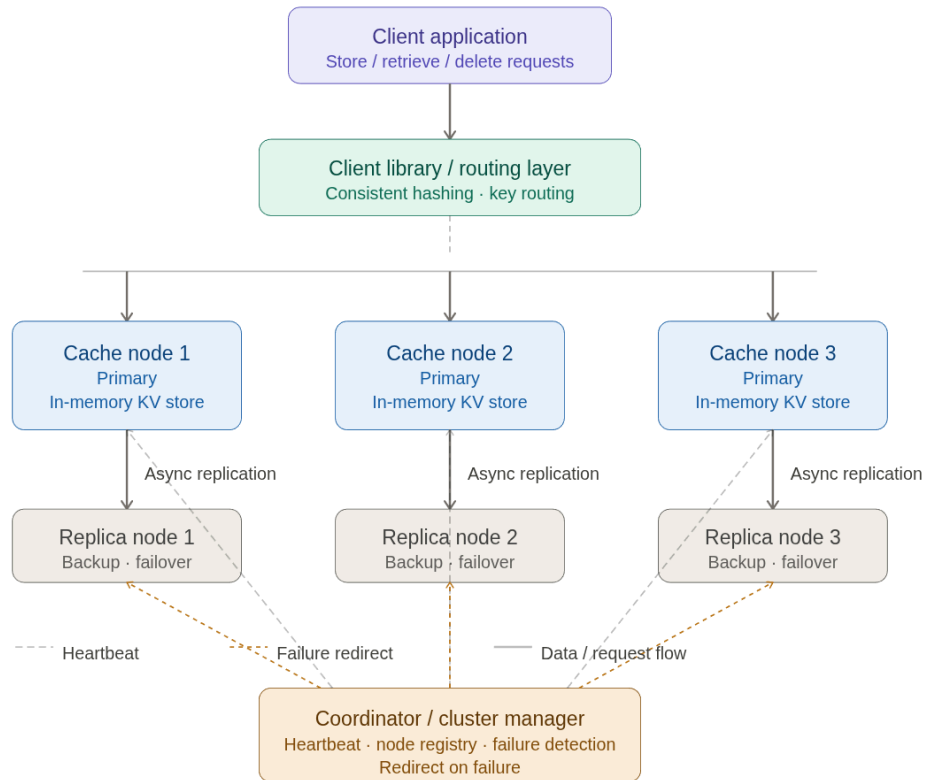
The specific objectives are:

- Implement an **in-memory key-value store** for efficient data storage and retrieval.
- Design a **distributed caching architecture** with multiple cache nodes.
- Implement **data partitioning using consistent hashing** to distribute data across nodes.
- Implement **replication mechanisms** to improve fault tolerance and availability.
- Implement **cache eviction policies** such as **LRU** and **TTL** for memory management.
- Provide a **client API** to interact with the distributed cache.
- Evaluate system performance using **latency and throughput metrics**.

3. Architecture

The proposed system consists of multiple cache nodes distributed across the network. A client application interacts with the cache through a routing layer that determines the correct node responsible for storing a given key using consistent hashing.

Each cache node maintains an in-memory key-value store and has an associated replica node that maintains a copy of its data for fault tolerance. A lightweight cluster manager monitors node health using heartbeat messages and handles node failures.



4. Features to be Implemented

The following core features will be implemented:

1. **In-Memory Key-Value Store** : Each cache node will store data in memory as key-value pairs, allowing fast retrieval and updates.
2. **Data Partitioning (Sharding)** : Keys will be distributed across multiple nodes using **consistent hashing**, ensuring balanced load distribution and scalability.
3. **Replication for Fault Tolerance** : Each primary cache node will have a replica node that maintains a copy of its data. If a primary node fails, the replica node can serve requests temporarily.
4. **Cache Eviction Policies** : Since the cache operates in memory, eviction strategies will be implemented:
 - **Least Recently Used (LRU)** to remove least accessed items.
 - **Time-To-Live (TTL)** to automatically expire old entries.

5. **Client API** : A simple API will allow applications to interact with the cache using operations such as:
 - PUT(key, value)
 - GET(key)
 - DELETE(key)
6. **Fault Detection** : Nodes will be monitored using heartbeat mechanisms to detect failures and redirect requests to replica nodes.

5. Technologies and Tools

- **Programming Language**: The system will be implemented using **Python** (or Go) due to its strong support for networking and rapid development.
- **Networking**: Communication between cache nodes and clients will be implemented using **REST APIs or gRPC**.
- **Data Storage**: Each cache node will use **in-memory hash maps/dictionaries** to store key-value pairs.
- **Serialization**: **JSON or Protocol Buffers** will be used for data serialization during communication between nodes.
- **Testing**: The system will be tested using **PyTest** (or Go's built-in testing framework).
- **Deployment**: **Docker containers** may be used to simulate multiple cache nodes in a distributed environment.

6. Tentative Timeline

Week	Tasks
Week 1	Finalize system design and architecture. Set up development environment and implement the basic in-memory key-value store with PUT, GET, and DELETE operations.
Week 2	Implement distributed architecture with multiple cache nodes and consistent hashing for data partitioning. Develop the client API for request routing.
Week 3	Implement replication between nodes and cache eviction policies such as LRU and TTL. Add heartbeat mechanism for basic fault detection.

Week 4	Integrate all components, perform testing and performance evaluation, simulate node failures, and prepare the final demonstration and documentation.
--------	--

7. Evaluation

The system will be evaluated based on performance and functionality.

Performance Metrics

- **Request Latency:** The expected latency for GET and PUT operations is $< 1s$ under normal load (local distributed setup using in-memory storage).
- **Throughput**
- **Scalability:** Performance should improve with the addition of nodes. For example, increasing from 1 to 3 nodes should show a noticeable improvement in throughput and reduced load per node.

Functional Evaluation

- Running multiple cache nodes simultaneously
- Demonstrating data distribution across nodes
- Demonstrating replication between nodes
- Simulating node failure and showing replica takeover
- Demonstrating eviction policies